

Proyecto ETL Forex

Informe de Entrega Final

Pipeline ETL end-to-end para datos del mercado forex, orquestado con Airflow, con modelo dimensional en PostgreSQL, streaming en Kafka y dashboards en Metabase y Streamlit.

Integrantes	Bryan Andres Herrera Betancur (2244008)
	Alessandro Yusty Ceballos (2240248)
Curso	ETL (G51)
Docente	Daniel Felipe Romero Bernal
Programa	Ingeniería de Datos e Inteligencia Artificial
Universidad	Universidad Autónoma de Occidente
Fecha	Mayo 2026

1. Resumen ejecutivo

Este proyecto implementa un pipeline ETL completo, reproducible y orquestado, que consume datos del mercado forex desde tres fuentes complementarias (Yahoo Finance, Finnhub vía WebSocket y Alpha Vantage), los valida con reglas declarativas de calidad y los consolida en un modelo dimensional sobre PostgreSQL. Los datos se exponen a través de dos dashboards complementarios: uno analítico construido sobre Metabase y otro operativo en tiempo real construido sobre Streamlit y Plotly, conectado al stream de Kafka.

Toda la infraestructura se levanta con un único comando (`docker compose up -d`), incluyendo nueve servicios: Postgres del proyecto, Postgres del metastore de Airflow, Zookeeper, Kafka, Airflow scheduler, Airflow webserver, Metabase, el producer de Kafka y el dashboard Streamlit.

Objetivos cumplidos

- Extracción desde 3 fuentes (Yahoo, Finnhub WebSocket, Alpha Vantage).
- Transformación a esquema unificado (timestamp, symbol, OHLC, source).
- Validación de calidad declarativa con bifurcación a cuarentena.
- Carga REAL al modelo dimensional (sin tareas fantasma).
- Kafka producer simulando CDC sobre la fact table.
- Kafka consumer + Streamlit con dashboard en tiempo real.
- Dashboard analítico en Metabase sobre la BD consolidada.
- Dos notebooks EDA (dataset y API).
- Documento técnico, README y .gitignore.

2. Atención a la retroalimentación de la entrega 2

Comentario del docente sobre la entrega 2:

«Muy incompleto el trabajo. Crearon muchas tareas fantasmas, que no tienen una conclusión real, por otro lado simulan el merge y la carga con bashoperator. La orquestación falla desde ahí. Las visualizaciones deben ser desde la base de datos consolidada, no desde las fuentes obtenidas.»

Observación del docente	Solución implementada
Tareas fantasma (Merge, validar_merge, verificar_etl, validar_etl)	Eliminadas. Solo permanecen tareas con efecto real en la BD.
Merge y carga simulados con BashOperator (echo)	Cargar_db reescrita como PythonOperator con inserciones reales vía SQLAlchemy
Orquestación falla desde la carga	DAG ETL_Delivery3 end-to-end funcional, idempotente. Cuarentena persiste en la BD
Visualizaciones desde las fuentes, no del warehouse	Metabase conecta a vw_quotes_flat. Producer de Kafka lee de fact_quotes (no de fact_quotes_flat)

3. Arquitectura

La arquitectura sigue un patrón Lambda simplificado donde un único modelo dimensional alimenta tanto el plano analítico (Metabase) como el plano operativo (Kafka + Streamlit), garantizando consistencia entre ambos dashboards y eliminando la redundancia de fuentes de verdad.

3.1. Stack tecnológico

Capa	Tecnología	Versión
Orquestación	Apache Airflow	2.9.2
Data Warehouse	PostgreSQL	15
Streaming	Apache Kafka (Confluent)	7.6.1
Validación	Reglas declarativas en pandas	-
BI analítico	Metabase	latest
Real-time dashboard	Streamlit + Plotly	latest
Contenerización	Docker Compose	v2
Lenguaje	Python	3.11

3.2. Flujo end-to-end

1. **preparar.py** genera los CSVs iniciales (yahoo.csv, finnhub.csv) tras elegir un par de divisas.
2. **Airflow DAG (ETL_Delivery3)** corre cada 2 minutos con tres ramas paralelas (Yahoo / Finnhub / Alpha). Cada rama: extrae → transforma → valida → carga.
3. **Kafka producer** consulta fact_quotes y publica al topic quotes_stream con un delay de 1 segundo entre mensajes, simulando CDC tal como pide el enunciado.
4. **Streamlit** consume el topic en background y grafica con Plotly. Buffer rodante de 2000 ticks.
5. **Metabase** conecta a la vista vw_quotes_flat (fact + dims) y permite construir dashboards drag-and-drop.

4. Modelo dimensional

Esquema en estrella sobre PostgreSQL, definido en sql/01_init_schema.sql. Se crea automáticamente la primera vez que arranca el contenedor de Postgres.

Tabla	Tipo	Rol
dim_symbol	Dimensión	Catálogo de pares (base/quote currency)
dim_source	Dimensión	Yahoo / Finnhub / Alpha. Precargada por seed.
dim_time	Dimensión	Timestamp desnormalizado (year, month, day, hour, minute, dow)
fact_quotes	Hechos	OHLC + price con FKs y UNIQUE(time, symbol, source)
quarantine_quotes	Soporte	Lotes rechazados con payload JSONB y razón
vw_quotes_flat	Vista	fact + dims joinado, listo para BI

Justificación del diseño

- Estrella en lugar de copo de nieve: las dimensiones son pequeñas y no requieren jerarquías profundas.

- UNIQUE(time, symbol, source) en fact_quotes habilita reejecuciones idempotentes del DAG.
- dim_source con seed precargado evita race conditions en la primera ejecución.
- Vista vw_quotes_flat se expone como punto de entrada único para BI, ocultando los joins.

5. Diseño del DAG (ETL_Delivery3)

El DAG está diseñado con tres ramas paralelas que convergen en una única tarea de carga al modelo dimensional. Cada rama es independiente: si una fuente falla validación, las otras continúan.

Árbol de dependencias

```
extraccion_yahoo → transformacion_yahoo → validar_yahoo ──┐
extraccion_finhub → transformacion_finhub → validar_finhub ──┴─> [cargar_db |
cuarentena]
extraccion_alpha → transformacion_alpha → validar_alpha ──┐
```

Tareas

Task ID	Tipo	Función
extraccion_yahoo / finhub / alpha	PythonOperator	Lee CSV o hace request a Alpha. 3 tareas paralelas.
transformacion_yahoo / finhub / alpha	PythonOperator	Normaliza al esquema unificado. Devuelve ruta vía XCom.
validar_yahoo / finhub / alpha	BranchPythonOperator	Aplica reglas de calidad. Decide rama: cargar_db o cuarentena.
cargar_db	PythonOperator	INSERT REAL en dim_* y fact_quotes vía SQLAlchemy.
cuarentena	PythonOperator	INSERT REAL en quarantine_quotes con payload JSONB.

Diferencias clave vs entrega 2

Aspecto	Entrega 2	Entrega final
cargar_db	BashOperator + echo	PythonOperator + SQLAlchemy
Merge	BashOperator (fantasma)	Eliminada
validar_merge	Sobre un echo	Eliminada
cuarentena	BashOperator + echo	PythonOperator + JSONB
Destino	SQLite, tabla por símbolo	PostgreSQL, esquema en estrella

6. Streaming con Kafka

Producer (app/producer.py). Implementa exactamente lo que el enunciado solicita: «a Python producer that iterates over the rows in your fact table and sends them to the Kafka topic with a slight time delay». Mantiene un cursor (último quote_id procesado) para no duplicar mensajes. Particiona por symbol para preservar orden. Reconexión automática a Kafka si falla.

Dashboard (app/dashboard.py). Thread daemon consume Kafka y llena un deque(maxlen=2000) thread-safe. Streamlit re-renderiza cada 2 segundos con un snapshot. UI con selector de símbolo, multiselect de fuentes, slider de ventana, 4 KPIs en vivo, gráfico Plotly con línea por fuente y tabla panorámica de todos los símbolos.

7. Dashboards

7.1. Dashboard analítico (Metabase)

Conecta a la BD trading_db y consume principalmente de la vista vw_quotes_flat. El proyecto incluye una biblioteca de 13 KPIs listos en docs/kpis.sql, adaptados al caso de uso (un par de divisas + tres fuentes).

Sección	KPIs incluidos
---------	----------------

Salud del pipeline	Total registros, fuentes activas, % éxito validación, cuarentena, última carga
Negocio (por fuente)	Evolución de close, estadísticas descriptivas, discrepancia Yahoo vs Finnhub
Temporales	Cobertura por día y fuente, volatilidad por hora, vela OHLC
Insights	Variación % diaria, distribución de precios

7.2. Dashboard real-time (Streamlit)

Disponible en <http://localhost:8501>. Conecta al topic Kafka quotes_stream y muestra: último precio + $\Delta\%$, máximo y mínimo de la ventana, gráfico Plotly con una traza por fuente, vista panorámica de todos los símbolos en buffer, expandir con los últimos ticks crudos.

8. Análisis exploratorio (EDA)

Notebook	Contenido
eda_dataset.ipynb	EDA del batch (Yahoo): calidad del dato, validación lógica OHLC, distribución del precio de cierre
eda_api.ipynb	EDA del stream (Finnhub): tasa de mensajes, inter-arrival times, justificación cuantitativa de la

9. Value Generation Articulation

9.1. Valor operacional (real-time)

El Kafka producer + consumer Streamlit soporta procesos operacionales donde la latencia importa: traders, sistemas de alertas o motores de decisión automatizados necesitan ver el precio actual con segundos de retardo. El topic `quotes_stream` actúa como un bus de eventos al que pueden conectarse múltiples consumidores. Por ejemplo, un servicio futuro de alertas que dispare notificaciones cuando un par cruce un umbral, o un microservicio de risk management. La arquitectura desacopla productores de consumidores, base de cualquier sistema reactivo en producción.

9.2. Valor analítico (BI)

El modelo dimensional + Metabase soporta procesos analíticos donde la profundidad histórica y la capacidad de cruzar dimensiones pesan más que la latencia. Permite responder preguntas como «¿cuál fue la volatilidad promedio del EUR/USD los lunes vs los viernes?» o «¿qué fuente reporta sistemáticamente precios más altos?». Es la base sobre la que se construirán los modelos predictivos futuros mencionados como objetivo del proyecto (ODS 1).

9.3. Convergencia

Crucialmente, ambos planos consumen del MISMO modelo dimensional: el producer de Kafka NO lee del stream crudo de Finnhub, sino de `fact_quotes` ya validada. Esto garantiza que el dashboard real-time muestra el mismo dato que el analítico (con el desfase de la simulación), eliminando la temida «verdad doble» típica de arquitecturas Lambda mal diseñadas.

10. Decisiones técnicas y trade-offs

Decisión	Alternativa	Trade-off
Postgres en lugar de SQLite	Mantener SQLite	+ Conexión nativa a BI / + concurrencia. – Más memoria.
Metabase para BI estático	Power BI / Looker	+ Open source, integra en compose. – Menos polish.
Streamlit para real-time	Dash / Bokeh	+ Más simple, hot-reload. – Threading limitado.
Producer lee de fact_quotes	Stream directo de Finnhub	+ Garantiza calidad validada. – No es CDC real.
Ramas paralelas del DAG	Pipeline secuencial	+ Throughput. – Más complejidad en branching.
Esquema dimensional	Tabla por símbolo (E2)	+ Soporta BI. – Requiere upserts en dims.
Validación pandas	Great Expectations 1.x+	+ Menos deps, más rápido. – Pierde ecosistema GE.

11. Entregables

Archivo / Carpeta	Contenido
docker-compose.yml	Stack completo de 9 servicios.
dags/ETL.py	DAG ETL_Delivery3 (carga real al modelo dimensional).
sql/01_init_schema.sql	Modelo dimensional (4 dims + 1 fact + cuarentena + vista).
sql/00_create_metabase_db.sql	Crea la BD que Metabase usa para sus metadatos.
app/producer.py	Kafka producer (simula CDC sobre fact_quotes).
app/consumer.py	Consumer standalone para debug.
app/dashboard.py	Dashboard Streamlit en tiempo real.
notebooks/eda_dataset.ipynb	EDA del batch (Yahoo).
notebooks/eda_api.ipynb	EDA del stream (Finnhub).
docs/technical_report.md	Documento técnico completo.
docs/kpis.sql	Biblioteca de 13 KPIs para Metabase.
docs/architecture.png	Diagrama de arquitectura.
README.md	Quick start, comandos útiles, decisiones clave.
.gitignore	Excluye venv, logs, secretos, CSVs intermedios.

12. Cómo ejecutar el proyecto

```
# 1. Clonar el repositorio
git clone https://github.com/<usuario>/ETL-Proyect.git
cd ETL-Proyect

# 2. Levantar el stack
docker compose up -d

# 3. Generar CSVs base
docker compose exec airflow-scheduler python /opt/airflow/preparar.py
# Opción 1 + índice de divisa (ej. 0 = EURUSD)

# 4. Activar el DAG en Airflow UI
# http://localhost:8080 (admin / admin)
```



```
# 5. Abrir los dashboards
# Metabase: http://localhost:3000
# Streamlit: http://localhost:8501
```

13. Próximos pasos

- Migrar `_PIP_ADDITIONAL_REQUIREMENTS` a un Dockerfile propio para imagen reproducible.
- Implementar un consumer Kafka que persista métricas agregadas en una tabla `fact_alerts`.
- Entrenar el modelo predictivo de variación cambiaria (objetivo asociado al ODS 1).
- Añadir CI con GitHub Actions: linter de Python + tests unitarios.
- Documentar runbook de operaciones (cómo monitorear, cómo escalar, cómo debuggear).